

SeedHammer II — the taproot pathological wallet

The hardest wallet this constellation describes, round-tripped on the machine.

This is the four-tier degrading vault — the wallet `CORPUS.md` calls *the* pathological example — expressed as a **depth-3 taproot tree**:

```
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,@0/48'/0'/0'/2'<0;1>/*,@1/48'/0'/1'/2'<0;1>/*,@2/48'/0'/2'/2'<0;1>/*))),{and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/48'/0'/3'/2'<0;1>/*,@4/48'/0'/0'/2'<0;1>/*,@5/48'/0'/1'/2'<0;1>/*))),{and_v(v:older(65535),multi_a(2,@6/48'/0'/2'/2'<0;1>/*,@7/48'/0'/3'/2'<0;1>/*)),and_v(v:older(4255898),multi_a(1,@8/48'/0'/0'/2'<0;1>/*,@9/48'/0'/1'/2'<0;1>/*,@10/48'/0'/2'/2'<0;1>/*))}}})
```

Two sha256 hashlocks. Both Bitcoin timelock flavours, absolute and relative. `multi_a` at thresholds 3, 2, 2 and 1. A **NUMS internal key**, so there is no key-path spend at all — every route to the coins goes through a script leaf. Eleven cosigners across **three masters**.

Until three commits ago this document could not exist. Every one of its four leaves is `and_v(v:...)` wrapping a timelock or a hashlock, and the device's tap-leaf reader named only `pk / multi_a / sortedmulti_a`. It returned *zero* leaves and the consent screen read "Complex policy — display only", while the host derived the same wallet's addresses without complaint. That was F-214; the fix replaced the vocabulary with an emitter, and this journey is the proof it worked.

The keys are BIP-39 test vectors. Never put funds behind them.

1 · Every slot's origin matches its own key file

The template carries eleven per-key origins, written by hand; the eleven key files declare them independently. If the two disagreed, the card would name a path the key does not live at — and **nothing downstream would notice**, because addresses come from the xpub a card carries, never from the origin it declares. That is F-217's lesson, and this gate is the answer to it.

```
@0 48'/0'/0'/2' ok
@1 48'/0'/1'/2' ok
@2 48'/0'/2'/2' ok
@3 48'/0'/3'/2' ok
@4 48'/0'/0'/2' ok
@5 48'/0'/1'/2' ok
@6 48'/0'/2'/2' ok
@7 48'/0'/3'/2' ok
@8 48'/0'/0'/2' ok
@9 48'/0'/1'/2' ok
@10 48'/0'/2'/2' ok
```

0 slot(s) disagree with their key file

2 · The engraved artifact

The keyless template set — 6 chunks. No --path: the origins ride in the template, and --path would flatten eleven distinct origins onto one.

And nothing would stop it.

An earlier draft of this page said md encode "refuses outright". Measured: it refuses the *keyed* form, where it can see one fingerprint bound to several different keys — but on this *keyless* template --path exits 0 and produces a perfectly good card with eleven wrong origins. The check needs a fingerprint *and* an xpub per slot, and a keyless template carries neither (F-221). **The card that gets engraved is the form the guard cannot see**, so leaving --path off here is a choice this journey makes, not one the tool enforces.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d54
7bfee9ace803ac0,{and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),mul
ti_a(3,@0/48'/0'/0'/2'<0;1>/*,@1/48'/0'/1'/2'<0;1>/*,@2/48'/0'/2'/2'<0;1>/*)),{and_v(v:after(1893456000),and_v(v:sha256
(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/48'/0'/3'/2'<0;1>/*,@4/48'/0'/0'/2'<0;1>
*,@5/48'/0'/1'/2'<0;1>/*))),{and_v(v:older(65535),multi_a(2,@6/48'/0'/2'/2'<0;1>/*,@7/48'/0'/3'/2'<0;1>/*)),and_v(v:older
(4255898),multi_a(1,@8/48'/0'/0'/2'<0;1>/*,@9/48'/0'/1'/2'<0;1>/*,@10/48'/0'/2'/2'<0;1>/*))}}) --fingerprint @0=73c5da0a
--fingerprint @1=73c5da0a --fingerprint @2=73c5da0a --fingerprint @3=73c5da0a --fingerprint @4=b8688df1 --fingerprint @5=b86
88df1 --fingerprint @6=b8688df1 --fingerprint @7=b8688df1 --fingerprint @8=28645006 --fingerprint @9=28645006 --fingerprint
@10=28645006 --group-size 0 --force-chunked
chunk-set-id: 0x5f855
md1ft7z4zs9zjtvyy5jmplrjttvyy49ykcqfw2fdssjjtvyyw2fdssj55jmpl9ef9kqhn9qydlzhdec
md1ft7z4zsgssjjtvyyw2fdssj54qxxqu2nxkcpapyqnxhdgfh8yp96hlypesq3ul02rhmtqcxc82mkn
md1ft7z4zs5x4mvaevc3h25sapudvvtas2de5z9dyppqzg2nxkmsm0vgqnxdgfh8yp9e85x45yge9ru
md1ft7z4zsew4e8cwwqy08m65x4mvaevc3h25sapudvvtas2de5z9dyq3rg52nxhqqqjze0u0fy087zm
md1ft7z4z3qllljgqt8fntsqpyuy6yqpgngxwxqw0za5zsh83w6pgnnchdq5deutks2fcet0ct6m96eu4
md1ft7z4z3ts6yd79dcdzxlz6ux3r030wrg3hcc9pj9qp5jsezs63gv3qvy2y2xugja08c9g
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

The engraved set is 6 md1 chunk(s).

3 · E1 — the cards re-encode to the policy they came from

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md verify --template tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,@0/48'/0'/0'/2'/<0;1>/*,@1/48'/0'/1'/2'/<0;1>/*,@2/48'/0'/2'/2'/<0;1>/*)),{and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/48'/0'/3'/2'/<0;1>/*,@4/48'/0'/0'/2'/<0;1>/*,@5/48'/0'/1'/2'/<0;1>/*)),{and_v(v:older(65535),multi_a(2,@6/48'/0'/2'/2'/<0;1>/*,@7/48'/0'/3'/2'/<0;1>/*)),and_v(v:older(4255898),multi_a(1,@8/48'/0'/0'/2'/<0;1>/*,@9/48'/0'/1'/2'/<0;1>/*,@10/48'/0'/2'/2'/<0;1>/*))})) --fingerprint @0=73c5da0a --fingerprint @1=73c5da0a --fingerprint @2=73c5da0a --fingerprint @3=73c5da0a --fingerprint @4=b8688df1 --fingerprint @5=b8688df1 --fingerprint @6=b8688df1 --fingerprint @7=b8688df1 --fingerprint @8=28645006 --fingerprint @9=28645006 --fingerprint @10=28645006 md1ft7z4zs9zjtvyvy5jmplrjttvyvy49ykcgfw2fdssjjtvyvy2fdssj55jmplp9ef9kqhn9qydlzhdec md1ft7z4zsgssjjtvyvyw2fdssj54qxxqu2nxkcpapyqnxhgdgh8yp96hlypesq3ul02rhmtqcx82mkn md1ft7z4zs5x4mvaevc3h25sapudvvtas2de5z9dyppqzg2nxkmsm0vgqnxhdgh8yp9e85x45yge9ru md1ft7z4zsew4e8cwvqy08m65x4mvaevc3h25sapudvvtas2de5z9dyq3rg52nxhqqqjze0u0fy087zm md1ft7z4z3qllljggt8fn tsqypuy6gqpgngxwxq0za5zsh83w6pgnnchdq5deutks2fcet0ct6m96eu4 md1ft7z4z3ts6yd79dcdzxlz6ux3r030wrg3hcc9p9p5jsezs63gv3gqv2y2xugja08c9g
OK
[exit 0]
```

Why md verify and not a md decode diff. Decode renders the per-key origins away: the text it returns is well-formed, missing the one field a signer needs to find its keys, and re-encodes to a *different card*. A journey asserting decode-equality would either fail on a perfectly correct plate or be quietly weakened until it passed. md verify exists for exactly this question, returns 0 or 1, and compares payloads rather than rendered text. The gap is filed as F-219.

4 · The bind — both card sets are the same wallet

The engraved set is keyless; the set the device gathers carries the eleven xpubs. Without this assertion the journey could engrave one wallet and prove another. The *template* id is key-stable, which is precisely what makes it the right thing to compare.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1ft7z4zs9zjtvyvy5jmplrjttvyvy49ykcgfw2fdssjjtvyvyw2fdssj55jmplp9ef9kqhn9qydlzhdec md1ft7z4zsgssjjtvyvyw2fdssj54qxxqu2nxkcpapyqnxhgdgh8yp96hlypesq3ul02rhmtqcx82mkn md1ft7z4zs5x4mvaevc3h25sapudvvtas2de5z9dyppqzg2nxkmsm0vgqnxhgdgh8yp9e85x45yge9ru md1ft7z4zsew4e8cwvqy08m65x4mvaevc3h25sapudvvtas2de5z9dyq3rg52nxhqqqjze0u0fy087zm md1ft7z4z3qllljggt8fntsqypuy6gqpgngxwxq0za5zsh83w6pgnnchdq5deutks2fcet0ct6m96eu4 md1ft7z4z3ts6yd79dcdzxlz6ux3r030wrg3hcc9p9p5jsezs63gv3gqv2y2xugja08c9g
template: tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*)),{and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*)),{and_v(v:older(65535),multi_a(2,@6/<0;1>/*,@7/<0;1>/*)),and_v(v:older(4255898),multi_a(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))}))
n: 11
wallet-policy-mode: false
md1-encoding-id: 5f855442464b572f581bb448b9f47ccd
wallet-descriptor-template-id: 44ad26a19b53048b6ff8957359a30c31
origins:
  @0: [73c5da0a/48'/0'/0'/2']
  @1: [73c5da0a/48'/0'/1'/2']
  @2: [73c5da0a/48'/0'/2'/2']
  @3: [73c5da0a/48'/0'/3'/2']
  @4: [b8688df1/48'/0'/0'/2']
  @5: [b8688df1/48'/0'/1'/2']
  @6: [b8688df1/48'/0'/2'/2']
  @7: [b8688df1/48'/0'/3'/2']
  @8: [28645006/48'/0'/0'/2']
  @9: [28645006/48'/0'/1'/2']
  @10: [28645006/48'/0'/2'/2']
wallet-policy-id: b03ab0477ffec5ea7780d8e1c706e2ff
wallet-policy-id-fingerprint: 0xb03ab047
note: stdout is a keyless descriptor template (no keys)
[exit 0]

$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fydqpts9zjtvyvy5jmplrjttvyvy49ykcgfw2fdssjjtvyvyw2fdssj55jmplp9ef9kzz2g9dxe2y7nujts md1fydqptsdkz289ykcgf22sqqrq9fntvqq7sjqfntk5ymnjqjatj0sucqg70h4gdtkemje3zptvqjvcnxxws md1fydqpts464y8g0rtzrm0vznwdq3tfqggqjz5e4kuxmmzqye4m2zdeeqfw4e8cwvqy08m65g5wj7tsrdu4lxh md1fydqpts6hdnh9nxa2jr583433dhkpfhxs g45szdydz32v6uqqq0llqey9n5e4cqzq7zdz2p56enkuv0wmmd md1fydqpt3qqz3xsuvuq97m9gpw0za5z3883w6pgmnchdq5jux3r03twrg3hckhp5gmuts62a ds9gszjdhkkm md1fydqpt3g3hcc9p9p5jsezs63gv3gqv9k6zqhwsv0jsk2rsl4egg2ep5859p875x67k9jqdtfexpz6x md1fydqpt3sxjz9m806prlsm7 94tkxqs6806lhaeh6reknlyagmwyjycf8044xtt9flsdlkv2cdfmwmvvtshs md1fydqpt3lf8tpwunu53n9svfgll7zvv5qknkgeal5wa4wy86389z3gq870g9 n6rjwuq7cms9xujhgxpd1943 md1fydqptjqqf99zcyrd98kjcuv8gdtu6gt04upy3z0n8ek4aj5kk9satjt7uv4am7jn7q2q8zxlcm12eczwj md1fydqptjdj ly68gp3qxznzrvapghktcljzr7j8mxxp87s87gwfzjsys8njyze0uj4852ajv5x9h80ejfs5f0ev md1fydqptjneynsu1328vme3ent5fcakske788nfhcc7nza0y nr9p5d3exan7jvup3kx8742qsqc0mpk87fkuj md1fydqptjmfhtfgqfec6yprg0dc3gdls899kx6k89kfakz2az67ajl8hy967je7sks39z8we5emvhw6cqrza md1fydqptnxmlcrst5m3s998x2rk7av9gmew82j3z5ap9302ynhp37gg6dz5u4emag0zr8gkyd3qwg763syq0m md1fydqptnvhsxwt3qdt2f9njdzjxzf3pk2mw xeza6dduqqnvtpl3vsgcj7vs58sl2hmlrny76x2a7hhfx8n5 md1fydqptns67znc3fa47zw97hmwk2vvhq2h6g9y9evngtkrskaxskvp4qchq5qpvp0pez9razz nwk32ujcyjk5 md1fydqptnanu69tqlftc3g6hj0nmpmk06drmsawv08c2v0vktv5t6qu9g4xkmv2dywrwkud7euvmc9egs7fl5 md1fydqpt5xgnj4chh02k9m3 a3x9a4h733cpg59kdh7306na4nhq32cd5w9c2uzlhxyrtzduftjem8nym63un md1fydqpt5fgpxkjsesxnnp7e00y36klmrkvghu772ljlrlwlhznccxfdd22zx6 72chhd5uajtcmvt0vz8hye793 md1fydqpt5hd0zup072vfew0h2t65xl3djg9z7s576y0hvw6a92khwlfwmwulc9zrupz2z7sktxueuey8thn md1fydqpt5l
```

```
737tj9hrdzaved93u3a2nawefnqlk8ycusz0quhfev0a3hqdgghjmksz3ry92dsr5uuvvw3jqhu9r md1fydqpt49pjhxf00s49ugex8rue307wdsLgcj5r8x9t5j5
4jfadzzykqz37sefnc0rhk5zckt9spyhvvhr3h md1fydqpt42amshp87fzcadzx07vs02d3t8efq2e50d8mg50ts8t8mquqwc90uyncTuc4sv9q4ytx0xzzyq4c
k md1fydqpt45jnl3rgupw7m2jr5caw7vxt9v7rn6wz3pd0pwa4xqjj8e7htel64394h6gme2g5uavvj00qjctyx md1fydqpt4a8ajc9sqw28jr665c4hq9t4ts
w75nz845y9up7y276lt29ywu3usza3aqrp2qqj37hvham46nhy
template: tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:after(1000000),and_v(v:sha256(a84dce4
0975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))),{and_v(v:after(1893
456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/<0;1>/*,@4/<0;1>/*,@5/
<0;1>/*))),{and_v(v:older(65535),multi_a(2,@6/<0;1>/*,@7/<0;1>/*)),and_v(v:older(4255898),multi_a(1,@8/<0;1>/*,@9/<0;1>/*,@1
0/<0;1>/*))}}})
n: 11
... clipped; full text in transcript_tr_pathological.txt
```

5 · What the device must prove to

Derived from the *cards*, not from the argument list that made them.

```
wallet idwallet-policy-id: 590f3abcaad2aca5a3f526917f5bb57a
receive 0..1bc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7pzzr
bc1p3wy7tcypvkm49wtyeq2yw4t0266vxdnq1kny5nsgzguc0fcvd13s5hkkp5
change 0..1bc1pnjrzcky5qq1hyz4rvhq5r96ca77wyccnfls0p517k8ae7402vjhshf3zya
bc1p7yrpv9q6j58vvh5yrnnj8ky6dnykn7z2lak4j0ckt52j84j8fxzs4h6khc
```

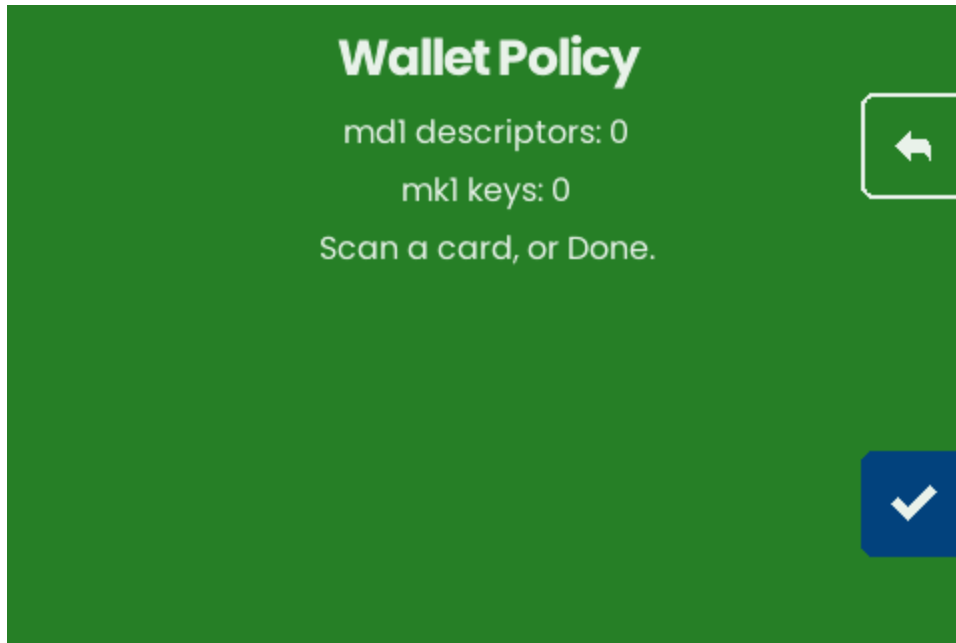
6 · The device



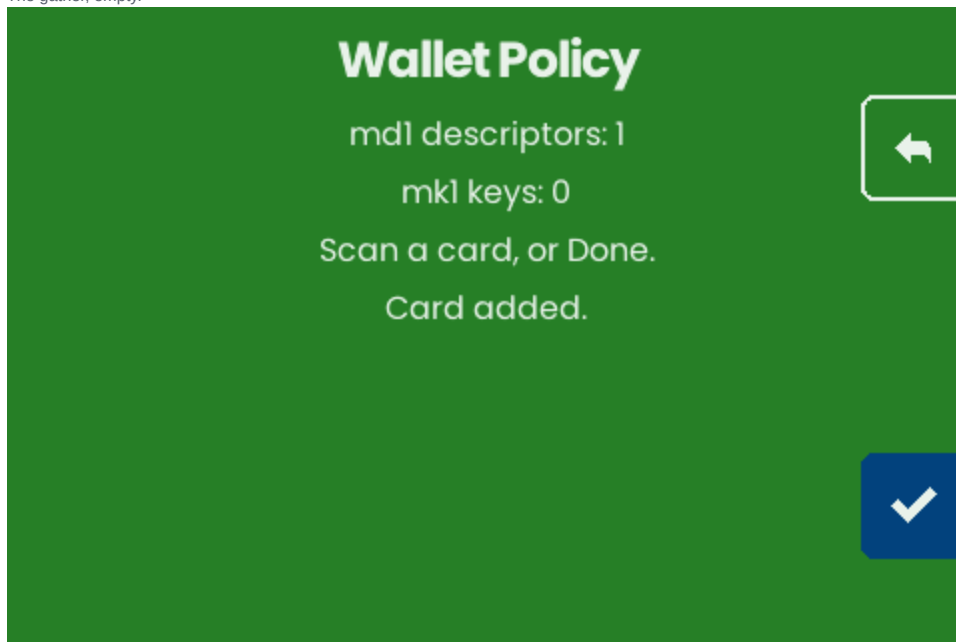
Home.



Wallet Policy.



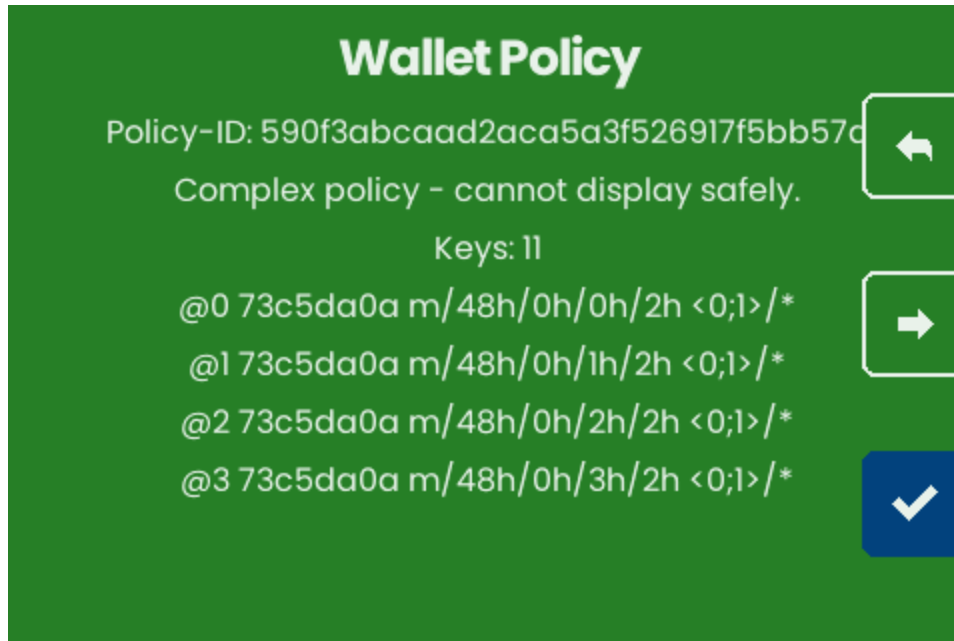
The gather, empty.



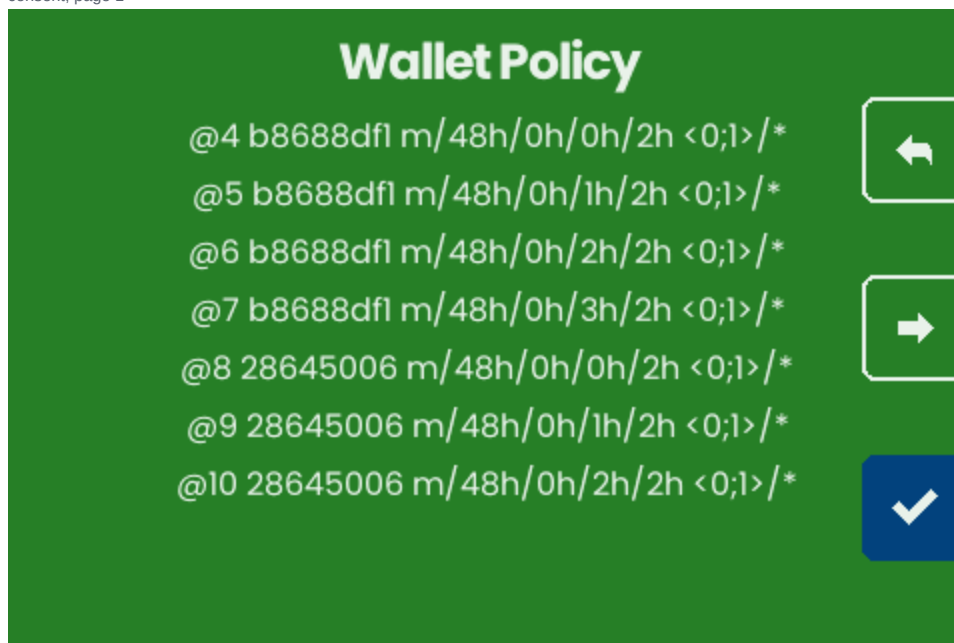
After all 24 chunks cross the reader: ONE card.

7 · The proof, on the consent path

Eleven key slots print before the addresses do, so the screen pages 4 times — and the walk reads every page, because a bound tuned for a four-key wallet would stop before reaching the half that matters.



consent, page 1



consent, page 2

Wallet Policy

Receive 0:
bc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7pzt

Receive 1:
bc1p3wy7tcypvkm49wtieq2yw4t0266vxdnqlkny5nsgzguc0fcvdl3s5hkkp5

Change 0:

←

→

✓

consent, page 3

Wallet Policy

bc1pnjrzc ky5qqlhyz4rvhq5r96ca77wyccnfls0p5l7k8advjhshf3zya

Change 1:
bc1p7yrpv9q6j58vwh5yrnnj8ky6dnykn7z2lak4j0ckt52xzs4h6khc

←

→

✓

consent, page 4

The key cards — the half a descriptor cannot supply

Everything up to here engraved a *descriptor*. A descriptor card is not a backup on its own: it names eleven @N slots and gives a cosigner no way to prove which slot is theirs. The mk1 key cards are that other half.

They bind to the **keyless template** card, not the keyed one. The stub is form-aware, so a card minted against the keyed policy carries a different stub and is refused against the template — **one wallet, two stubs**. The template is what gets engraved here, so the template is what the cards bind to, and the journey fails if they do not:

```
$ head -2 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/keys.txt
[73c5da0a/48'/0'/0'/2']xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwCHbF5XB2mSTexchZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyx
UFL4r6KFrF
[73c5da0a/48'/0'/1'/2']xpub6DzhyrnFFYQ1HimDiM388xHnDiRPNdZJFBmmxge3Y1WwCHLTMJLfRuhRHqnQCPbtJ3fGKTuKFLHzzwpJkp5Dtc3UtLKZKaVZe
lyqMBXd6Vk
[exit 0]

note: stdout is watch-only – public keys only, cannot spend
$ /scratch/code/shibboleth/mnemonic-engrave/design/journeys/mk_cards_from_json.py /scratch/code/shibboleth/mnemonic-engrave/
design/journeys/out/tr-pathological/mk-encode.json /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-patholog
ical/mk-encode-raw.txt
11 key cards -> 30 mk1 chunks
[exit 0]

template-id prefix: 44ad26a1  policy-id prefix: b03ab047  stub mk embedded: 44ad26a1
The cards bind to the keyless template. Same wallet, correct form.
```

The plate checklist

What actually gets cut — the descriptor chunks and every key card in one set, so the plate count is the real one rather than the descriptor's alone:

```
The engraved set: 6 mdl + 30 mk1 = 36 strings.
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me bundle --in /scratch/code/shibboleth/mnemonic-engrave/design/j
ourneys/out/tr-pathological/backup-strings.txt --preview /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pa
thological/plates --png --manifest /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/manifest.js
on
me: rendered plate 1 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-1.png
me: rendered plate 2 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-2.png
me: rendered plate 3 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-3.png
me: rendered plate 4 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-4.png
me: rendered plate 5 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-5.png
me: rendered plate 6 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-6.png
me: rendered plate 7 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-7.png
me: rendered plate 8 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-8.png
me: rendered plate 9 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-9.png
me: rendered plate 10 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-10.png
me: rendered plate 11 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-11.png
me: rendered plate 12 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-12.png
me: rendered plate 13 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-13.png
me: rendered plate 14 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-14.png
me: rendered plate 15 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-15.png
me: rendered plate 16 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-16.png
me: rendered plate 17 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-17.png
me: rendered plate 18 -> /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/tr-pathological/plates/plate-18.png
... clipped; full text in transcript_tr_pathological.txt
```

The seed, and the refusal that holds

The one plate that is not public. me refuses an ms1 outright — it will not render, preview or transmit a secret — so the seed plate is counted in the manifest and never drawn. That refusal is what makes the rest of the bundle safe to preview at all:

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-pathological/seeds/master-A.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
[exit 0]

$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms encode --phrase abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon about
ms10e ntrsqq qqqqqq qqqqqq qqqqqq qqqqqq qqqqqq qqcj9 sxraq 34v7f
word count: 12
language: english (BIP-39 checksum valid)
passphrase: not stored in ms1 (record separately if used)
warning: stdout carries private key material (can spend) – redirect or encrypt (e.g. '> file.txt' or '| age -e ...')
[exit 0]
```

```
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --in /dev/fd/63 --hex
me: refusing to emit msl over NFC: msl is secret seed entropy and must never be transmitted by radio (RF eavesdropping risk). Enter it by hand on the device: New > Input Seed > CODEX32.
... clipped; full text in transcript_tr_pathological.txt
```

The restore test — and why taproot needs its own

Section 4 proved the cards *re-encode* to their own policy. That is a transcription check on the bytes. This asks the question a backup actually has to answer: **hold these plates and the three seeds, and do you get the wallet back?**

Taproot makes that harder than it is for wsh. A `tr()` address commits to the **taptree shape** as well as to the keys, so a restore can recover all eleven keys, rebuild the tree wrongly, and produce a wallet that looks correct and is not. The test therefore ends on **addresses**, not on keys — get the shape wrong and they differ even when every key is right.

It restores only from what a holder of the plates has: each slot's origin is read *from the card*, the key is derived from its seed at that origin, the template is re-annotated with the card's own origins, and the addresses come from the result:

```
$ /scratch/code/shibboleth/mnemonic-engrave/design/journeys/restore_test_tr_pathological.py
Restoring 11 slots from the engraved template + the seeds:
@0 73c5da0a at 48'/0'/0'/2' -> xpub6DkFAXWQ2dHxq2vatrt9...
@1 73c5da0a at 48'/0'/1'/2' -> xpub6DzhyrnFFYQ1HiMDiM38...
@2 73c5da0a at 48'/0'/2'/2' -> xpub6EGx8sPr9FxPPE1rbZaz...
@3 73c5da0a at 48'/0'/3'/2' -> xpub6E6Z3Ss5TXJYNJp4U1q3...
@4 b8688df1 at 48'/0'/0'/2' -> xpub6FQya7zGhR92kacYsNnj...
@5 b8688df1 at 48'/0'/1'/2' -> xpub6EAMBjLn1jiQuajTsNRk...
@6 b8688df1 at 48'/0'/2'/2' -> xpub6EGceQV5wHrRemjYkJyk...
@7 b8688df1 at 48'/0'/3'/2' -> xpub6ELyn7moeEZ9hJ1HmSm6...
@8 28645006 at 48'/0'/0'/2' -> xpub6DnEBNkSJKBYQmsbhS1s...
@9 28645006 at 48'/0'/1'/2' -> xpub6F6gx8ZP9R3R3eYsU2Pe...
@10 28645006 at 48'/0'/2'/2' -> xpub6Dh4twkBRkzxDSau8Fdh...

receive addresses
MATCH bc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7p2r
MATCH bc1p3wy7tcypvkm49wtieq2yw4t0266vxdnqlkny5nsgzguc0fcvdl3s5hkkp5

change addresses
MATCH bc1pnjrzcky5qqlhyz4rvhq5r96ca77wyccnfls0p5l7k8ae7402vjhshf3zya
MATCH bc1p7yrpv9q6j58vwh5yrnnj8ky6dnykn7z2lak4j0ckt52j84j8fxzs4h6khc

RESTORED: 11 of 11 slots, and the rebuilt wallet spends to the SAME
addresses as the engraved card – so the taptree shape survived too.
[exit 0]
```

Verified to be capable of failing.

Swapping two slots' recovered keys — the same key set, a different assignment, so the same tree shape over different leaves — makes the rebuilt wallet spend to entirely different addresses and the test fails loudly. A restore test that cannot fail proves nothing, and this one was checked rather than assumed.

8 · What this run established

The capture is a **comparison**. It is handed the host's id and addresses and throws if the screen disagrees, and the absence assertions reject "display only" and "no addresses" outright — so the device cannot pass by refusing politely.

```
chunks presented24
cards gathered1
consent pages read4
wallet id matched590f3abcaad2aca5a3f526917f5bb57a
addresses matchedbc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7p2r
bc1p3wy7tcypvkm49wtyeq2yw4t0266vxdnqlkny5nsgzguc0fcvd13s5hkkp5
bc1pnjrzccky5qqlhyz4rvhq5r96ca77wyccnfls0p5l7k8ae7402vjhshf3zya
bc1p7yrpv9q6j58vwh5yrnnj8ky6dnykn7z2lak4j0ckt52j84j8fxzs4h6khc
```

The negative control is a command, not a memory. `python3 capture_tr_pathological.py --prove-it-can-fail` corrupts one character of one expected address and exits 0 only if the walk *caught* it. A comparison nobody has made fail is not evidence that it can.

Reproducing

```
bash transcript_tr_pathological.sh > transcript_tr_pathological.txt 2>&1
python3 capture_tr_pathological.py                # rebuilds emu.wasm, drives it
python3 capture_tr_pathological.py --prove-it-can-fail # the negative control
python3 build_pdf_tr_pathological.py                # this document
```

What this backup will NOT do

Tiers 1–2 cannot be spent from these plates alone. Both need the 32-byte preimage of the sha256 hashlock. **No plate carries it** — record it separately, with the vault, or those tiers are decoration.

The tiers do not open in the order they read. Tier 4 (1-of-3) matures at ~365 days, tier 3 (2-of-2) at ~455 — the weakest key-set unlocks ~90 days first. Each `older()` clock runs per-coin from that coin's confirmation, not from the day you engraved.

In master terms this is a 1-of-3 vault with delays, not an 11-key multisig. Three of the four tiers need only one master: **A** alone for tier 1 (unlocked since 2016, but only with the secret word), **C** alone for tier 4 after ~365 days with no word, and **B** alone for tier 3 after ~455. Only tier 2 needs two, and it is the furthest out. The vault survives losing any two seed plates and falls to the theft of any one, given time.

What this journey does NOT show

- **No spend, and for tiers 1–2 no spend is possible from this backup at all.** Both hashlock tiers need the 32-byte preimage to spend, and *zero* backup strings carry it (F-132).
- **The tiers do not degrade in the order they read.** Tier 4 (1-of-3) matures at 365 days; tier 3 (2-of-2) at ~455 — the weakest key-set unlocks ~90 days *first* (F-133).
- **No plate read-back.** There is no reader; the card-set equality above is the honest proxy.
- **No ms1 secret leg.** Taproot-independent, and already shown in the earlier journeys.
- **Decode's output still does not round-trip** (F-219). The card carries its origins and both `md inspect` and `md decode` now report them — an earlier draft of this line said neither did, which its own section 4 already disproved. What remains is that decode's *stdout* renders them away, so re-encoding what it prints yields a different card.